# Übungsblatt 3: Threads, Präemption und zeitgesteuertes Warten

**Musterlösung · Echtzeitsysteme (EZS)**

Prof. Dr.-Ing. Daniel Klünder

## Aufgabe 1: Zurückkehren, warten oder weiterrechnen? (6 Punkte)

Eine Sortierstation soll alle 200 ms eine neue Messung bearbeiten.

1. Im kooperativen Programm wird `MessungStep()` aus der Hauptschleife aufgerufen. In einem RTOS-Projekt gibt es stattdessen den Thread `Messung(void *argument)`. Wo darf eine Endlosschleife stehen und wie kommt jeweils die nächste Aufgabe zum Zug? (2 Punkte)
2. Der RTOS-Thread ruft `osDelay(7)` auf. Geben Sie seine Zustandsfolge bis zum nächsten Lauf an. Welche Einheit hat die Sieben und warum garantiert das Ablaufen der Wartezeit noch keinen sofortigen Start? (2 Punkte)
3. Jemand ersetzt den Aufruf durch die übliche `HAL_Delay(7)`. Wird der Thread dadurch Blocked? Können höher priorisierte Threads und zugelassene Interrupts trotzdem ausgeführt werden? (2 Punkte)

**Musterlösung:** `MessungStep` muss nach endlicher Arbeit zurückkehren. Der Dispatcher ruft dann einen anderen bereiten Schritt auf. Die dauerhafte Schleife liegt im kooperativen Hauptprogramm; der Anwendungsfortschritt wird ausdrücklich gespeichert. Ein RTOS-Thread darf seine Arbeits-/Warteschritte in einer Endlosschleife wiederholen: Der Kernel sichert den Kontext mit eigenem Stack und kann höhere bereite Threads auswählen. Ohne echtes Warten kann ein hochpriorer Thread trotzdem niedrigere aushungern.

Bei `osDelay(7)` gilt Running → Blocked → Ready → Running. Sieben bezeichnet Kernelticks. Nach Ablauf muss der Scheduler erst zuteilen; höhere bereite Prioritäten können den Start verzögern.

Die übliche HAL-Funktion wartet aktiv und macht den Thread nicht durch die Kernel-Warteverwaltung Blocked. Höhere bereite Threads können ihn präemptieren und nicht gesperrte Interrupts laufen. HAL-Millisekunden und Kernelticks sind getrennte Zeitbasen. Je zwei Punkte für die begründeten Antworten.

## Aufgabe 2: Drei endliche Arbeiten auf einem Kern (10 Punkte)

Für eine Paketwaage gelten feste Prioritäten und Präemption. Alle API-, Interrupt- und Wechselkosten sind null. Die Tabelle enthält Freigaben und eigene CPU-Zeiten, keine Boardmessungen.

| Thread | Priorität | Freigaben / Verhalten | CPU-Zeit pro Job |
|---|---|---|---|
| Ausschleusen | höchste | wartet bis 3 ms; ein Job, danach wieder warten | 2 ms |
| Wiegen | mittlere | periodisch bei 0, 6, 12 ms; nach jedem Job bis zur nächsten Freigabe warten | 2 ms |
| Etikettbericht | niedrigste | ein Job bei 0; nach Abschluss warten | 5 ms |

Für Wiegen gilt die relative Deadline $D = 3\,\text{ms}$. Der Ausschleusethread ist bei null bereits Blocked; andere Arbeit fehlt. Leerlauf wird als Idle bezeichnet.

1. Zeichnen Sie die CPU-Belegung im halboffenen Intervall $[0, 15)\,\text{ms}$. Kennzeichnen Sie die Ursachen der Wechsel. (4 Punkte)
2. Nennen Sie die Zustände der drei Threads bei 2,5 und 3,5 ms. (2 Punkte)
3. Berechnen Sie die CPU-Zeitanteile einschließlich Idle in diesem Intervall und die Antwortzeiten aller drei Wiegejobs. (2 Punkte)
4. Ausschleusen erhält zusätzlich einen zweiten Job bei 6 ms. Wann startet und endet der ebenfalls bei 6 ms freigegebene Wiegejob? Hält er seine Deadline? (2 Punkte)

**Musterlösung:**

| Intervall in ms | CPU | Ursache am Anfang |
|---|---|---|
| $[0, 2)$ | Wiegen | höchste bereite Priorität |
| $[2, 3)$ | Etikettbericht | Wiegen wartet |
| $[3, 5)$ | Ausschleusen | Ereignis macht höchsten Thread bereit |
| $[5, 6)$ | Etikettbericht | Ausschleusen wartet wieder |
| $[6, 8)$ | Wiegen | periodische Freigabe |
| $[8, 11)$ | Etikettbericht | Wiegen wartet |
| $[11, 12)$ | Idle | Bericht fertig, alle drei warten |
| $[12, 14)$ | Wiegen | periodische Freigabe |
| $[14, 15)$ | Idle | Wiegen wartet |

Bei 2,5 ms: Ausschleusen Blocked, Wiegen Blocked, Etikettbericht Running. Bei 3,5 ms: Ausschleusen Running, Wiegen Blocked, Etikettbericht Ready. Präemption ist kein sachliches Blockieren des Berichts.

Wiegen erhält 6 ms, Ausschleusen 2 ms, Etikettbericht 5 ms und Idle 2 ms, zusammen 15 ms. Die Zeitanteile sind entsprechend $6/15 = 40\,\%$, $2/15 \approx 13{,}33\,\%$, $5/15 \approx 33{,}33\,\%$ und $2/15 \approx 13{,}33\,\%$. Die Wiegejobs enden bei 2, 8 und 14 ms und haben jeweils $R = 2\,\text{ms} \leq D$.

Mit dem zusätzlichen Ereignis läuft Ausschleusen von 6 bis 8, Wiegen von 8 bis 10 ms. Für den Wiegejob gilt $R = 10 - 6 = 4\,\text{ms} > 3\,\text{ms}$; seine absolute Deadline 9 ms wird verfehlt. Bereitschaft bei 6 ms ist kein Start bei 6 ms.

## Aufgabe 3: Ein Tickplan mit und ohne Drift (8 Punkte)

1. Ein Kernel hat $f_{\text{tick}} = 250\,\text{Hz}$. Berechnen Sie Tickperiode und aufgerundete Tickzahlen für 6 ms und 18 ms. Warum garantiert `osDelay(1)` keine physische Mindestwartezeit von einer ganzen Tickperiode? (3 Punkte)
2. Für diese Teilaufgabe gilt stattdessen ein 1-kHz-Tick. Drei Arbeitsphasen benötigen nacheinander 3, 1 und 5 ms. Der erste Start ist bei null, alle Arbeitsenden liegen genau auf Tickgrenzen; weitere Störungen fehlen. Bestimmen Sie die nächsten drei Starts bei „Arbeit, dann `osDelay(12)`“ und bei einem absoluten 12-Tick-Sollraster. (3 Punkte)
3. Eine Arbeit endet bei Tick 53; der nächste geplante Start bei Tick 48 ist bereits verpasst. Die Periode beträgt 12 Ticks. Geben Sie eine geeignete Reaktion ohne sofortige Nachholserie an. Was kann ein Zähler verpasster Warteziele aussagen, was ohne zusätzliche Deadlinedefinition nicht? (2 Punkte)

**Musterlösung:** $T_{\text{tick}} = 1/250\,\text{s} = 4\,\text{ms}$. Mit $n = \lceil t f_{\text{tick}}/1000 \rceil$ ergeben sich für 6 ms zwei und für 18 ms fünf Ticks. Ein Aufruf kurz vor der nächsten Tickgrenze kann bereits an dieser Grenze wieder Ready werden. Die anschließende Zuteilung kann zusätzlich verzögert sein; die Umrechnung allein garantiert keine physische Mindestwartezeit.

Relative Starts: $0 + 3 + 12 = 15$, $15 + 1 + 12 = 28$ und $28 + 5 + 12 = 45\,\text{ms}$. Absolute Starts im ungestörten Modell: 12, 24 und 36 ms. Das absolute Raster addiert die Arbeitszeit nicht auf jede Periode; es erzwingt unter Last keinen pünktlichen tatsächlichen Start.

Eine zulässige Politik zählt die Zielverfehlung, richtet das nächste Ziel auf $53 + 12 = 65$ aus und blockiert bis dorthin. Alternativ kann ein spezifizierter Entwurf veraltete Aktivierungen überspringen. Eine sofortige Serie wäre für viele Ausgänge unerwünscht und erzeugte zusätzliche Last. Der Zähler zeigt verpasste Warteziele, beweist aber ohne passenden Job-/Deadlinevertrag weder Anzahl noch Ausbleiben aller Deadline-Misses.

## Aufgabe 4: Ressourcen und falsche Abhilfen (8 Punkte)

1. Drei CMSIS-Threads erhalten Stacks mit 768, 1536 und 640 Byte. Pro Thread werden zusätzlich 96 Byte Verwaltungsdaten angenommen. Berechnen Sie die Summe. Welche weiteren Anteile fehlen für eine vollständige Speicherbilanz? Wie viele Byte sind 192 native `StackType_t`-Elemente zu je vier Byte? (3 Punkte)
2. Ein Diagnose-Thread steht über einer Ausgabesteuerung, bleibt ständig Ready und ruft in seiner Rechenschleife `osThreadYield()` auf. Die Ausgabesteuerung bleibt ebenfalls Ready. Wird sie dadurch ausgewählt? Benennen Sie das Problem und eine geeignete Änderung. Es gibt keine gemeinsam gehaltene Ressource. (3 Punkte)
3. Die logische Interruptschwelle für Kernelaufrufe ist 6; vier Prioritätsbits und passende Gruppierung sind vorausgesetzt. Darf ein IRQ mit logischer Priorität 4 `osThreadFlagsSet` aufrufen? Darf ein hierfür zulässiger IRQ mit Priorität 7 `osThreadFlagsWait` aufrufen? (2 Punkte)

**Musterlösung:** $768 + 1536 + 640 + 3 \cdot 96 = 3232\,\text{Byte}$. Idle-/Timer-Task, weitere Objekte, Allokationszuschläge und Ausrichtung fehlen. Die TCB-Größe ist hier nur angenommen. Für den Gesamt-RAM-Bedarf sind auch Handlerstack, weitere Daten und das konkrete Speicherlayout zu beachten; im Heap enthaltene Blöcke nicht nochmals außerhalb zählen. Native Elemente: $192 \cdot 4 = 768\,\text{Byte}$. CMSIS `stack_size` ist bereits in Byte angegeben.

Yield macht den Diagnose-Thread nicht Blocked. Er bleibt der höchste bereite Thread; die Ausgabe verhungert (Starvation). Echtes Warten auf einen nächsten Termin oder ein Ereignis

kann Zuteilung ermöglichen; auch ein fachlich begründeter niedrigerer Diagnoserang ist möglich. Ohne Ressourcenabhängigkeit liegt keine Priority Inversion vor.

IRQ-Priorität 4 ist dringlicher als die erlaubte Schwelle 6 und darf in dieser Konfiguration keine Kernel-API aufrufen. Priorität 7 ist für passende nichtblockierende Kernelaufrufe erlaubt. `osThreadFlagsWait` ist jedoch ein Threadaufruf und auch aus diesem IRQ unzulässig. Der IRQ darf einen Hinweis mit `osThreadFlagsSet` übergeben, nicht selbst blockierend darauf warten.

## Aufgabe 5: Wie viele Hinweise bleiben erhalten?

Ein Thread wartet auf Bit 2, also die Maske `0x04U`. Ein zulässiger anderer Thread setzt dieses Bit bei 2, 3 und 9 ms. Der Empfänger ruft `osThreadFlagsWait(0x04U, osFlagsWaitAny, osWaitForever)` bei 5 und 11 ms und direkt nach dem zweiten erfolgreichen Wait noch einmal auf. Vorher ist das Bit gelöscht, weitere Set-Aufrufe fehlen. Erfolgreiche Waits löschen das angenommene Bit automatisch.

1. Welche beiden Waits kehren sofort zurück und was passiert beim dritten?
2. Wie viele Set-Aufrufe und wie viele erfolgreiche Waits gibt es? Warum sind diese Zahlen verschieden?
3. Wie erkennen Sie einen Fehler im Rückgabewert einer Flagfunktion? Weshalb wäre ein Vergleich mit `osOK` falsch?

**Musterlösung:** Bei 5 ms ist das Bit durch die Set-Aufrufe bei 2 und 3 ms gesetzt. Der Wait kehrt sofort zurück und löscht es. Bei 11 ms wird das bei 9 ms gesetzte Bit angenommen und gelöscht. Der dritte Wait blockiert, da kein Hinweis mehr aussteht.

Drei Set-Aufrufe ergeben zwei erfolgreiche Waits. Wiederholtes Setzen eines bereits gesetzten Bits speichert nur einen ausstehenden Hinweis, keinen Ereigniszähler. Flagfunktionen liefern Flagbits oder einen Fehlerwert mit gesetztem `osFlagsError`-Bit. Fehlerprüfung: `(flags & osFlagsError) != 0U`. `osOK` ist der Statuscode anderer APIs und kein Test für erfolgreich gelieferte Flagbits.

## Aufgabe 6: HeavyMath, Heartbeat und Tasterreaktion mit FreeRTOS

**Programmieren, etwa 35–50 Minuten.** Implementieren Sie das Beispiel aus Vorlesung 3 auf dem **NUCLEO-H755ZI-Q / STM32H755ZIT6** mit STM32CubeMX und VS Code mit STM32 VS Code Extension. Es läuft ausschließlich der **M4-Kern**, auch für Systemstart und Taktkonfiguration; M7 bleibt deaktiviert. Die Projektvorbereitung erfolgt vor dem Termin. Benötigt werden nur die eingebauten LEDs und B1.

| Threadfunktion | CMSIS-Priorität | Gefordertes Verhalten |
|---|---|---|
| `Lab03_Event` | `osPriorityHigh` | auf Tasterhinweis warten, LD1/PB0 umschalten |
| `Lab03_Heartbeat` | `osPriorityNormal` | LD2/PE1 sofort, dann alle 500 Kernelticks umschalten |
| `Lab03_HeavyMath` | `osPriorityBelowNormal` | ununterbrochen rechnen, ständig Ready bleiben |

Die vorbereitete Basis nutzt **FreeRTOS, CMSIS-RTOS2, Präemption und 32-Bit-Ticks mit 1000 Hz**. `lab03_rtos.c/.h` erzeugt die drei Threads und stellt GPIO-, IRQ- und Wartehilfen bereit. Implementieren Sie die drei Funktionsrümpfe in `code/lab03/threads_student.c` selbst; entfernen Sie anschließend `WaitForImplementation`.

1. **Heartbeat:** Initialisieren Sie ein lokales `next` mit `osKernelGetTickCount()`. Rufen Sie in einer Endlosschleife `Lab03_ToggleHeartbeat()` und danach `Lab03_WaitHeartbeat(&next)` auf. Die Wartehilfe blockiert bis zum nächsten absoluten 500-Tick-Ziel und richtet sich bei einer Zielverfehlung neu aus. Welche vollständige Blinkperiode ergibt sich im ungestörten Modell? Warum darf hier kein `HAL_Delay` stehen?
2. **HeavyMath:** Aktualisieren Sie in einer Endlosschleife den vorbereiteten `volatile uint32_t lab03_accumulator` mit `lab03_accumulator * 1664525U + 1013904223U`. Verwenden Sie weder Delay noch Yield oder Interruptsperre. Die einfache Rechnung dient als dauerhafte CPU-Last; ihr Ergebnis wird fachlich nicht benötigt. Warum können Heartbeat und Tasterreaktion trotzdem ausgeführt werden?
3. **Tasterreaktion:** Rufen Sie einmal `Lab03_EnableButtonIRQ()` auf. Warten Sie anschließend in einer Endlosschleife mit `osThreadFlagsWait(LAB03_BUTTON_FLAG, osFlagsWaitAny, osWaitForever)`. Prüfen Sie das Fehlerbit `osFlagsError`; im Fehlerfall `Lab03_Fail()`. Erhöhen Sie nach jedem erfolgreichen Wait `lab03_event_responses` und schalten Sie mit `Lab03_ToggleEvent()` LD1 um. Zwischen Hinweisen soll der Thread Blocked sein.
4. **IRQ-Weg erklären:** B1/PC13 löst über die vorbereitete HAL-Callbackkette `Lab03_ButtonIRQ()` aus; diese setzt das Thread-Flag. Beschreiben Sie den Weg vom Tastendruck bis zur LED-Aktion. Warum setzt die ISR nur einen Hinweis und warum wird der IRQ erst nach dem Kernelstart im Ereignis-Thread freigegeben?

5. **Prüfen:** Bauen und laden Sie nur das CM4-Ziel. Prüfen Sie die folgenden Fälle und protokollieren Sie Erwartung und Beobachtung. Lesen Sie Diagnosewerte gemeinsam nur bei angehaltenem M4; ein Debughalt ist keine Zeitmessung.

| Versuch | Zu prüfende Beobachtung |
|---|---|
| Grundzustand ohne B1 | LD2 blinkt; `lab03_accumulator` verändert sich trotz Heartbeat weiter |
| B1 während HeavyMath | LD1 reagiert; `lab03_event_responses` steigt; LD2 blinkt weiter; `lab03_irq_errors` bleibt null |
| HeavyMath höher priorisieren | Vorhersage aus Teilaufgabe 6 mit dem Board vergleichen |

6. **Starvation selbst erzeugen:** Ändern Sie ausschließlich die HeavyMath-Priorität in `Lab03_Start()` von `osPriorityBelowNormal` auf `osPriorityAboveNormal`. Ereignis-Thread bleibt `osPriorityHigh`. Sagen Sie vor dem Neustart voraus, was mit beiden LEDs passiert. Begründen Sie, weshalb auch ein `osThreadYield()` in HeavyMath den Heartbeat nicht zuverlässig zum Zug bringt. Stellen Sie danach die ursprüngliche Priorität wieder her.

**Ergebnis:** drei eigene Threadfunktionen, Prüfprotokoll und Erklärung der Präemption. Tasterprellen kann mehrere Flanken erzeugen; ein Thread-Flag speichert nur einen ausstehenden Hinweis. Ein Tastendruck ist daher weder eine garantierte einzelne Umschaltung noch ein verlustfrei gezähltes Ereignis. Eine Entprellung ist hier kein Pflichtteil. Ohne Boardlauf dessen Status angeben.

**Quellen:** Buttazzo, *Hard Real-Time Computing Systems*; Arm CMSIS-RTOS2; ST UM2408, RM0399 und STM32CubeH7, Template *BootCM4_CM7Gated* (hier ohne Freigabe von CM7). Papierzeiten und Speichergrößen sind Lehrannahmen.

**Musterlösung Heartbeat:**

```
#include "lab03_rtos.h"

void Lab03_Heartbeat(void *argument)
{
    (void)argument;
    uint32_t next = osKernelGetTickCount();
    for (;;) {
        Lab03_ToggleHeartbeat();
        Lab03_WaitHeartbeat(&next);
    }
}
```

Bei 1000 Hz entsprechen 500 Ticks nominell 500 ms. Umschalten alle 500 Ticks ergibt eine vollständige Hell-Dunkel-Periode von 1000 Ticks, also nominell einer Sekunde. Die erste Umschaltung erfolgt beim ersten tatsächlichen Lauf des Threads. Das absolute Raster vermeidet das fortlaufende Aufaddieren der Arbeitszeit, garantiert aber keine verzögerungsfreie Zuteilung. `HAL_Delay` würde den Thread aktiv warten lassen und HeavyMath Rechenzeit nehmen.

**Musterlösung HeavyMath:**

```
void Lab03_HeavyMath(void *argument)
{
    (void)argument;
    for (;;) {
        lab03_accumulator = lab03_accumulator * 1664525U
                          + 1013904223U;
    }
}
```

Die Rechnung erfolgt vorzeichenlos modulo $2^{32}$. `volatile` sorgt für beobachtbare Speicherzugriffe, ist jedoch keine Datensynchronisation. Nur HeavyMath schreibt den Akkumulator. Dieser Thread blockiert nie; der präemptive Scheduler kann ihm trotzdem die CPU entziehen, sobald ein höher priorisierter Thread bereit ist. Dass der Idle-Thread bei dauerbereiter Rechenlast nicht läuft, ist in diesem Versuch beabsichtigt.

**Musterlösung Tasterreaktion:**

```
void Lab03_Event(void *argument)
{
    (void)argument;
    Lab03_EnableButtonIRQ();
    for (;;) {
        const uint32_t flags = osThreadFlagsWait(
            LAB03_BUTTON_FLAG, osFlagsWaitAny, osWaitForever);
        if ((flags & osFlagsError) != 0U) {
            Lab03_Fail();
        }
        ++lab03_event_responses;
        Lab03_ToggleEvent();
    }
}
```

Die vorbereitete Kette lautet: PC13-Flanke, CM4-Handler `EXTI15_10_IRQHandler`, `HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_13)`, `HAL_GPIO_EXTI_Callback`, `Lab03_ButtonIRQ`, `osThreadFlagsSet`. Die ISR blockiert nicht und übernimmt keine Rechen- oder Blinkschleife. Sie kann einen wartenden Ereignis-Thread Ready machen. Der CMSIS-/FreeRTOS-Adapter fordert den nötigen Kontextwechsel an; nach dem ISR-Ende darf der nun höchstpriorisierte Thread HeavyMath verdrängen. Ein bereits gesetztes Flag wird beim Wait ohne erneutes Warten angenommen und dabei automatisch gelöscht.

Alle drei Handles sind vor `osKernelStart()` angelegt und geprüft. Der IRQ bleibt zunächst gesperrt. Erst der laufende Ereignis-Thread verwirft alte EXTI-/NVIC-Anforderungen und aktiviert ihn. Danach wird kein Hinweisbit blind gelöscht; eine neue Flanke zwischen Aktivierung und erstem Wait bleibt als Hinweis erhalten. Die gewählte Startpolitik ignoriert bewusst Flanken vor dieser Freigabe.

| Zustand | Heartbeat | Tasterreaktion | HeavyMath |
|---|---|---|---|
| Grundkonfiguration | kurz Running, dann Blocked bis zum Sollziel | Blocked am Flag; bei Hinweis kurz Running | Running, solange kein höherer Thread bereit ist; sonst Ready |
| HeavyMath auf AboveNormal | bleibt nach Freigabe Ready, kann aber nicht laufen | verdrängt weiterhin HeavyMath; wartet danach wieder | erhält alle übrige Threadrechenzeit |

Im zweiten Versuch kann LD2 schon vor der ersten Umschaltung stehen bleiben, weil HeavyMath von Beginn an über dem Heartbeat liegt. LD1 reagiert weiterhin auf angenommene Hinweise. Auch `osThreadYield()` macht HeavyMath nicht Blocked; der höchstpriorisierte bereite Thread bleibt HeavyMath. Abhilfe schaffen das Wiederherstellen der niedrigeren Priorität oder fachlich passende blockierende Arbeitspausen. Es liegt Starvation vor, keine Priority Inversion durch eine Ressource.

`lab03_heartbeat_edges` zählt Umschaltungen, `lab03_event_responses` erfolgreiche Flagannahmen und `lab03_target_misses` erkannte verpasste Warteziele. Ein verhungernder Heartbeat führt seine Wartehilfe nicht aus: Ein unveränderter Zielzähler beweist deshalb keine eingehaltenen Fristen. Ein LED-Test belegt Funktion, keine numerisch garantierte Antwortzeit. Ein Flag zählt Prellflanken nicht verlustfrei; für die Prüfung wird keine Eins-zu-eins-Zuordnung zwischen Tastendruck und LED-Wechsel gefordert.

**Prüfkriterien:** eigene Schleifen aller drei Threads, echtes Blockieren in Heartbeat und Ereignis-Thread, dauerbereite HeavyMath-Schleife, richtige Prioritäten und Flagfehlerprüfung, keine Interruptsperre und kein aktives Warten in den beiden höher priorisierten Threads. CM7 bleibt auch beim Start und während des Debuggens deaktiviert.